

Specifications

Current intended behavior and current system structure.

Working draft

Current intended behavior and current system structure.

Product Surface

Scope today

The product currently has two visible layers:

- a public OWA website shell for orientation and placeholder public pages
- a dedicated Nexus shell under /nexus/* for the first guest-facing OWA Nexus slice

Implemented surface today includes:

- public Home, About, and Docs pages inside the public shell
- Nexus identity routes for sign-in, create, claim, restore, and security
- a locality creation route for canonical geographic assembly creation
- Nexus routes for Dashboard, Discussions, Votes, Roles, Trust, and Library
- a shell-level Packet Explorer overlay
- a hidden wrapper-level /nexus/account custody route reached from the profile area

Not implemented today:

- remote multi-node sync beyond the local SQLite packet store
- real-time chat
- protected or private spaces
- trust-weighted ranking, moderation enforcement, delegation, or proposal execution
- dedicated routed packet detail pages outside Packet Explorer

Public docs surface

- the public /docs route renders a docs hero, a document directory, a selected readable document panel, and downloads/resources panels
- the current directory includes Charter, Nexus README, Implementation Guide, Specifications, and Roadmap

- readable docs are served from app/public/generated/public-docs.generated.ts
- Markdown and PDF downloads are served from generated artifacts under /downloads/*
- the readable document surface uses one continuous document with an outline and anchored sections

Public content links

- public content actions are represented by PublicPageAction in app/public/content-types.ts
- actions may target internal public routes, external URLs, or static download paths under /downloads/
- existing internal route href usage remains supported as a compatibility bridge while newer content should prefer explicit target objects

Current surface roles

Library

Library is the scoped browse surface.

- it is packet-backed
- it defaults to a local-only view keyed to the active authority scope
- Open packet launches Packet Explorer without navigating away from Library
- packet actions on Library cards remain intentionally narrow

Packet Explorer

Packet Explorer is the inspect-and-traverse surface.

- it opens as a shell-level overlay
- it is still read-only for packet mutation, but its Home workspace now includes live search, import, and export workbenches
- packet validation is now a first-class Explorer workflow rather than only a future concept
- it is separate from Library
- it supports session-persistent tabs with an auto-created Home tab
- its Home workspace now splits into Search, Import, and Export sub-tabs rendered directly in the shared inspector row
- Search is now a manual grouped packet finder over the current preferred packet index, with All, Direct, Name, and Text result groupings, capped preview slices inside All, and paged category views for deeper result review

- Import now supports two-step packet and bundle ingest through Analyze then Commit, with paste-first JSON input plus web-only .json file upload
- Import now also supports three explicit validation modes: Validate first, Validate after, and Don't validate
- Import now exposes recent import-report history and direct Open report follow-up actions using the existing import_report packet type, with the current history view intentionally reflecting the latest local report per imported artifact identity rather than a full append-only event ledger
- Export now supports raw preferred-revision packet export, bounded bundle export, full local-store bundle export, a compact direct packet lookup when no export target is preloaded, and an explicit cancel/reset path when a packet export target was preloaded earlier
- it now uses shared packet display-title normalization instead of exposing raw encoded packet ids as loaded titles where a human title can be derived
- it supports View as inspection lenses for Summary, Raw, Adapted, and Read Model
- it exposes Data, Lineage, Verification, Links, and Actions inspector rails using the same compact attached-tab style as the Home workspace tabs
- its packet tab deck now presents Home as the leftmost workspace tab, uses middle truncation for packet labels, and exposes desktop hover metadata for full packet titles and revision context
- its main content region now reads as one primary scroll surface inside the drawer instead of trapping long packet reads inside tiny inner scroll panes
- its packet-shell and inspector-control header bands can now collapse locally to reclaim vertical space without leaving helper rows behind, and collapsed bands are restored from the top command row
- it groups Incoming and Outgoing links by related packet, with explicit Open in new tab, Open in current tab, and View in Library actions for graph traversal
- it surfaces read-only NexusActionState and NexusActionIntentDescriptor data where available
- its verification rail now distinguishes structural validity, compatibility, signature result, signer result, provenance, local trust, and local versus external report history
- its verification rail now includes a direct Verify or Reverify control, refreshes the active packet payload after local validation, and explicitly surfaces whether the latest local report is current for the preferred revision or stale against newer packet revisions
- packet search rows and export lookup rows now surface cached verification state when available so untrusted or failed packets are visible before opening the inspector

Discussions

Discussions are the most mature interactive Nexus surface today.

- discussion feed, thread, and post workspaces are packet-backed
- reply, vote, expand, and create actions are now runtime-projected instead of being only page-local heuristics
- interactive writes use the shared prepare/sign/finalize mutation corridor
- compatibility still keeps legacy discussion type shapes readable while canonical discussion packets drive current writes

Dashboard

Dashboard is now a real packet-backed preview workspace rather than only a shell landing page.

- it loads scope-backed metrics, queue items, and preview packet projections from runtime payloads
- preview sections now use the shared packet-card and action-list language rather than route-local one-off cards
- one packet can be focused at a time inside each preview surface, and that focus state remains local to the surface rather than becoming a routed detail view
- focused packet actions still use the same shared packet action menu language as non-focused preview rows
- packet action menus now include Validate, Revalidate, and View verification where local or imported verification evidence exists

Locality creation

`/nexus/locality/create` is now a guided review-first locality workflow rather than a thin write form.

- the route still searches the existing locality directory first
- when search does not find the intended locality, the existing `create_candidate` seam now seeds the broad-to-narrow builder directly instead of dead-ending
- top-level search can now still hand off into locality creation when same-name results exist elsewhere, so a result like Ontario in one branch does not suppress creating Ontario under another chosen parent path
- the create UI can still be edited target-first, but preview and create now normalize that editing state into a broad-to-narrow planner path before runtime planning
- the builder now uses a real non-mutating Review path step before any write occurs
- locality rows are now descriptor-first under the hood: each row can carry a hierarchy system, local type label, local type key, and legacy compatibility bucket
- sparse paths are now valid planner truth, so a path such as Canada -> Vancouver no longer needs a required intermediate region row just to satisfy the old ladder

- review distinguishes reused existing levels from newly planned locality levels
- duplicate warnings are now actionable through Use existing, Edit name, and Create anyway
- Use as home locality is now an explicit toggle shared by both existing-locality selection and newly created locality completion
- review can also show a pre-checked home-branch inclusion checklist, but that checklist remains preview-only in this pass and does not yet change persisted home-locality semantics
- created locality packets now record descriptor metadata inside linked `Location.spatial_payload`, while legacy nation | region | city | district levels remain compatibility buckets rather than the full future locality model
- locality confirm now writes through one composite runtime apply seam above `locality.path.create`, so locality graph creation, home-locality selection, association, follow, and temporary main-tree display choices are orchestrated together instead of being chained one mutation at a time in the page

Scope sidebar

The Nexus scope sidebar now reflects the packet-native scope graph more directly.

- it groups scope context into Main tree, Home scopes, Associated scopes, Followed scopes, and Discoverable scopes
- Home scopes render as a projected trunk ordered from the broadest mounted geography down toward the personal You scope
- associated scopes now count as mounted related scopes, but remain outside the geographic home trunk rather than being mixed into its ancestry chain
- associated, followed, main, and discoverable scopes are now server-projected grouped sections instead of thin client-side reconstructions from id arrays
- associated and followed sections can now persist lightweight parent-context display independently, using text-only contextual chains rather than full extra scope cards
- Main tree defaults to the home scope trunk when no explicit visibility preference has been saved, then uses explicit packet-backed display preferences for add/remove actions over eligible home, associated, and followed scopes
- scope rows stay click-to-open, while secondary actions move into compact side overflow menus
- section placement now carries most of the relationship meaning, rather than depending on inline badge clutter
- follow and association actions now use the canonical packet-native mutation corridor and then refresh shell projection from the server result
- guests are routed through the existing identity/auth gate instead of silently writing follow or association state

Shared packet-card action UI

Several packet-facing surfaces now share one compact card and row action language.

- dashboard preview rows, focused packet panels, and related Nexus packet lists now use reusable packet action menus instead of route-local action clusters
- compact icon badges can surface packet state without consuming card body space, and expose tooltip detail when needed
- Open in Explorer is now part of the shared packet action vocabulary rather than a one-off dashboard affordance
- validation actions now use the same shared packet action language, with local modal feedback and a deep-link path into Explorer verification

Votes

Votes are currently read-only.

- the route is packet-backed
- guests can inspect governance visibility cues and proposal or vote cards
- voting, objection, delegation, and execution workflows are not yet live
- visible disabled actions now use the same shared feature-status explainer pattern rather than acting like dead buttons

Current caveats

The following behaviors are not active product truth yet:

- initiative filtering in feeds, Library, or Explorer
- official versus unofficial packet visibility modes
- automoderation or visibility projection beyond current placeholders and packet-backed evidence reads
- projection definitions now drive preferred packet-action surface routing, but they do not yet fully generate generic UI layouts
- available actions are partly runtime-projected, not fully definition-driven
- policy-driven voting and decision execution are still not active
- universal UI component extraction and loading-overlay integration are ongoing

Nexus Routes And Workflows

Public routes

- / = landing page for OWA
- /about = public explanation page
- /docs = public charter destination page and source-material explainer
- /login = redirect to /nexus/identity/sign-in
- /signup = redirect to /nexus/identity/create

Nexus routes

- /nexus redirects to /nexus/dashboard
- /nexus/dashboard = packet-backed dashboard
- /nexus/discussions = packet-backed discussion shell with Feed, Thread, and Post
- /nexus/votes = read-only vote floor
- /nexus/library = scoped packet browse surface
- /nexus/trust = scoped trust posture and relationship surface
- /nexus/roles = scope-centric role review surface
- /nexus/account = hidden wrapper-level custody route
- /nexus/identity/* = sign-in, create, claim, restore, and security ceremonies
- /nexus/locality/create = guided locality directory and creation flow

Core workflows

Nexus entry

1. User enters /nexus. 2. The route redirects to /nexus/dashboard. 3. The shell loads in Global Guest state. 4. The initial mounted baseline is Global + You.

Scope selection

1. User opens the grouped scope sidebar. 2. The shell updates the active scope lens. 3. Dashboard, discussions, votes, roles, trust, and Library re-project against that scope.

Dashboard review

1. Dashboard loads packet-backed metrics, queue items, and preview packet lists for the active scope. 2. Clicking a packet row opens its preferred surface or falls back to Explorer when no route surface is appropriate. 3. Packet action menus can focus a packet inside the local preview section or open it in Explorer. 4. Validation actions can run local packet verification, return a compact result modal, and then deep-link into Explorer verification when needed. 5. Focus remains a surface-local interaction rather than a separate route or shell-wide mode.

Discussion posting

1. The route loads packet-backed discussion state. 2. The actor is a real Element(kind: "person"), including temporary guests. 3. The browser prepares a canonical mutation through the shared prepare/sign/finalize mutation corridor. 4. The active web identity shell signs the prepared packets locally. 5. Finalize re-verifies digest, signature, proof level, authority, and policy before persistence.

Trust and role review

1. Trust loads scope-local trust posture and relationship state. 2. Roles loads exact-scope role claims, claimants, and support or dispute evidence. 3. Protected guest actions open the shared auth-gate flow instead of trying raw writes.

Locality search and creation

1. User opens /nexus/locality/create. 2. The route searches the existing locality directory first and still allows selecting an existing locality immediately. 3. If search does not find the intended place, create_candidate can seed the builder directly, including from the top search input when Enter is pressed. 4. If same-name locality results exist elsewhere in Nexus, top-level search can still route into the create flow instead of forcing reuse from the wrong branch. 5. The builder may be edited as one lowest-locality row plus optional parent rows, but the adapter normalizes that state into one broad-to-narrow ordered path before preview or create. 6. The user runs Review path, which now previews the canonical planner without writing packets. 7. Planner ancestry now follows the ordered path itself rather than the old fixed ladder, so sparse paths like Canada -> Vancouver are valid current behavior. 8. Review shows which levels would reuse existing localities, which levels would create new locality packets, and any duplicate warnings that still need a decision. 9. Review rows and search results now expose descriptor-aware locality labels when linked Location metadata is available, while legacy levels remain compatibility buckets. 10. Duplicate warnings can now route back into the builder through Use existing, Edit name, or forward into an explicit Create anyway path. 11. Confirm now routes through one composite locality.graph.apply runtime seam, which reruns the structural locality planner, writes locality packets, applies selected home or association or follow relations, stores temporary main-tree display preferences, and then returns refreshed shell graph data. 12. If Use as home locality is on, selecting an existing locality or creating a new one both apply the canonical relation.residence.add path inside that composite apply flow. 13. The review-only home-branch checklist can be adjusted visually in this pass, but it does not yet change stored home-locality packet semantics.

Packet inspection

1. User opens Packet Explorer from the shell, Library, or link traversal. 2. Explorer loads packet summary, raw data, adapted data, read model, lineage, verification, grouped links, and runtime action visibility. 3. Explorer stays read-only for packet mutation, but its Home workspace now includes live Search, Import, and Export workbenches. 4. Search and export lookup rows now surface cached verification state when local or imported reports exist. 5. Explorer verification shows local summary state, imported or external report history, explicit current-versus-stale revision anchoring for the latest local report, and the layered breakdown that keeps structural validity, cryptographic validity, provenance, and local trust separate. 6. Library packet highlighting now clears itself when the highlighted packet is no longer represented by an Explorer packet tab.

Packet import and validation

1. User opens Packet Explorer Home and selects Import. 2. The import workspace accepts pasted JSON or a browser .json file. 3. Analyze can run in one of three modes: Validate first, Validate after, or Don't validate. 4. Validate first runs structural checks plus packet verification before commit, and blocks commit on invalid signatures or canonicalization mismatches. 5. Validate after commits structurally safe packets first, then writes verification reports and local cache state immediately afterward. 6. Don't validate keeps the import structural-only, but still records a local import report for the artifact/session. 7. The Import workspace keeps recent import_report history visible so the operator can reopen a report or jump into one affected packet without rerunning the whole import flow. 8. That Import History surface currently represents the latest local report per imported artifact identity, not a complete append-only import event ledger.

Packet validation

1. A packet action menu can expose Validate, Revalidate, or View verification depending on current local and imported evidence. 2. Validation runs through a runtime verification service rather than the user-authored mutation corridor. 3. The local runtime signs verificationreport and importreport packets with its own validator identity and also updates a local fast-read verification index. 4. Imported external verification reports are readable in Explorer, but local trusted status still comes only from the local node's own validation reports in this phase. 5. The Explorer Verification rail now exposes a direct Verify / Reverify control that refreshes the current packet payload after a local validation pass. 6. If a later preferred revision replaces the revision a local report targeted, that older report remains visible as history but is surfaced as stale rather than current verification state.

Shell behavior

The Nexus shell currently provides:

- a dedicated Nexus layout separate from the public shell
- a two-column left-side navigation model on desktop
- a mobile overlay tray that opens the real menu rails immediately, uses Open menu as the visible trigger label, shrinks with rail collapse, and closes when both rails are minimized
- a desktop Packet Explorer drawer that keeps its width session-persistent and uses a dedicated drag-resize seam
- a session-scoped early-access welcome gate that blocks shell interaction until dismissed
- function-first versus scope-first as a shell preference
- You as a first-class personal scope lens
- grouped Home, Associated, Followed, Main, and Discoverable scope sections with compact row-level overflow menus

- server-projected scope sections, including independent temporary preferences for Main visibility and parent-context display in the associated and followed sections
- independent rail collapse state
- a profile-area route into account and identity custody surfaces

Architecture And State

Active source split

- core/* holds portable packet logic, schemas, builders, interpreters, contracts, and pure projections
- runtime/* holds trusted runtime coordination, storage adapters, runtime services, query services, auth/trust/discussion orchestration, and API-facing glue
- app/* holds application-layer components, hooks, constants, public content, and shared shell state
- src/app/* holds the Expo Router route shell and API entrypoints

Current runtime and state patterns

- public pages are still mostly stateless
- Nexus state is shared through local React context providers
- shell and route projections load from packet-backed API routes
- the runtime store is SQLite-backed
- packet parsing runs through type compatibility and adaptation instead of route-local patches
- Packet Explorer session state is shell-level rather than route-level page state

Packet and compatibility foundations in active use

Current packet behavior in code includes:

- PacketEnvelope = { header, body }
- stable packet_id
- immutable revision_id
- multi-parent parentrevisionrefs
- type schema_version
- type revision_mode
- raw stored packets preserved as historical fact
- adapted runtime packets used as the normal read shape

- target-version-aware compatibility reads for supported types

Forward ontology currently active in code includes:

- Action, Relation, and Location as first-class packet types, with Action(subtype: initiative) carrying the forward initiative/default anchor
- Report as a first-class packet type for verification and import reporting
- Element.subtype as the forward classifier, with kind preserved as compatibility metadata
- widened Claim packets for packet-targeted assertion content plus optional relation_assertion
- widened Reaction packets with canonical subtype semantics
- Policy.relation_requirements as the packet-backed seam for relation support rules

Current scope consumer direction in code includes:

- shell scope projection now prefers packet-native Relation(subtype: defaultancestryparent) for ancestry
- home locality now resolves through relation-first projection with explicit compatibility fallback instead of claim-first shell logic
- follow now resolves packet-natively from Relation(subtype: follow) first, with legacy shell preference state reduced to an explicit compatibility bridge
- association now resolves packet-natively from Relation(subtype: association) first, with legacy claim-only reads reduced to explicit compatibility projection and associated scopes treated as mounted related scopes
- followed, associated, known, discoverable, and home-ancestor semantics are now carried as additive packet-backed scope-summary metadata
- locality creation now emits provisional Location(subtype: region) packets plus Relation(subtype: definedbylocation) so linked location packets are part of the live writer path as well as the read seam
- the active shell sidebar now consumes server-projected graph sections rather than reconstructing section truth from thin scope-id arrays
- scope projection now returns home, associated, followed, main, and discoverable sections, with descriptor-aware grouping and optional lightweight parent-context chains for associated and followed scopes
- locality confirmation now uses one composite runtime apply seam above locality.path.create so structural locality writes, scope relations, and packet-backed claimed display preferences are coordinated together
- main is a display preference, not a relation; claimed actors now persist it through Preference.element packets with the legacy runtime table kept as a compatibility cache
- claimed shell preference reads are session-bound before projecting private Preference.element state
- guest scope-display preference behavior remains an explicit compatibility/session bridge

- Preference.element.value.interface.shell_chrome now carries live navigation mode, theme mode, and UI density reads/writes through the signed mutation corridor for claimed actors, with the shell preferences API reserved for guest compatibility state
- the current home trunk renders broadest-to-smallest, while associated scopes remain mounted related scopes outside the geographic ancestry chain
- shell UI actions for follow and association now route through the canonical mutation corridor and refresh scope projection after success instead of relying on local-only toggles
- dashboard payloads are now scope-backed runtime projections rather than static shell filler
- shared packet-card projection and action-menu behavior now spans dashboard previews and related Nexus surfaces
- focused packet state is currently a surface-local UI layer over packet-backed preview lists rather than a separate routed or core packet concept
- the first verification chapter now treats report packets as the signed source of truth for local packet verification and import reporting, with a runtime-owned verification cache layered on top for fast UI reads
- the local runtime now owns a normal signed validator identity and uses it to sign verificationreport and importreport packets without routing those writes through user-authored mutation tickets
- OWA-specific initiative-anchor relation-policy lookup now lives in a narrower adapter layer instead of being embedded directly inside the generic scope-graph projection core

Public docs build system

- docs/public/public-docs.manifest.json is the source list for generated public docs
- scripts/validate-public-docs.mjs validates manifest and source-of-truth rules before generation
- scripts/build-public-docs.mjs compiles readable docs data into app/public/generated/public-docs.generated.ts
- the same build emits Markdown and PDF downloads under public/downloads/ and version records under docs/public/version-records/
- the chaptered internal docs use chapter files as canonical source material; the top-level shell docs are not part of the public compiled document content

Current runtime contracts worth keeping visible here:

- NexusActionState
- NexusActionIntentDescriptor
- NexusPacketExplorerPayload

Query and surface boundaries

- packet storage, compatibility, import/export, and merge remain below the route layer
- route components consume query payloads rather than storage classes directly
- runtime projects discussion and Explorer action visibility rather than leaving those rules entirely in page code

Current naming and structure notes

- route file names are lowercase and match path segments directly
- React components use PascalCase
- route screens continue to end with Page
- shared Nexus components live under app/components/nexus

The current repo is no longer using the older domain or storage root names. The active architecture is the core / runtime / app / src/app split described above.

Trusted coordinator structure

Trusted runtime coordinator code now lives under runtime/trusted_coordinators/*. The folder contains shared coordinator result helpers, the scaffold manifest, the Trusted Dispatch Coordinator front desk, the compatibility Trusted Request Coordinator implementation, the portable-resolution coordinator, the packet-projection coordinator, the Trusted Definition Coordinator, the Trusted Regulation Coordinator, the Trusted Planning Coordinator, the Trusted Building Coordinator, the Trusted Inspection Coordinator, the Trusted Certification Coordinator, direct packet workflow promotion, composite workflow promotion, and composite workflow adapter descriptors. Nexus server routes and mutation services may call these coordinators, but executable trusted behavior remains runtime-owned rather than imported from packet definitions. Foldered trusted coordinators are gated: external callers use public coordinator surfaces while internal function modules and registries stay private. The scaffold audit records which coordinators are fully foldered and which remain legacy-flat. Building is now foldered/gated and consumes trusted operation plans to emit candidate packet graphs; it does not own definition lookup, policy resolution, default resolution, dependency resolution, certification, or archival. Inspection is now foldered/gated and validates build results against frozen operation plan snapshots. Certification is now foldered/gated and owns the ticket/signature/hash handoff after Inspection, while Archive remains responsible for packet-store writes later.

The core packet-definition surface now includes richer projection descriptors: field bindings, layout/component keys, preferred surfaces, action registry keys, dependency IDs, policy action IDs, and resolver preset IDs. Generic packet definitions currently provide summary-card and detail-panel projection descriptors for active packet types. The packet action service uses preferred projection surfaces for focus/open routing. Full UI layout generation from projection descriptors is not yet complete.

Known Gaps And Provisional Notes

Current known gaps

- identity/auth and discussions remain the two strongest end-to-end verticals
- votes, trust, roles, and Library are packet-backed, but are still comparatively provisional
- dashboard is live and packet-backed, but deeper verification, trust, and governance meaning on its preview surfaces remains provisional
- role creation and editing remain unsupported
- broader moderation, trust weighting, and governance execution remain unsupported

Location and relationship caveats

- residence is now the packet-backed source of mounted geographic ancestry, with canonical writes producing a relation without automatic claim wrapping
- identity.location_disclosure remains optional profile metadata rather than mounted-scope truth
- association remains distinct from residence
- followed scopes are now packet-native on the write path, but legacy shell-preference follow remain readable through an explicit compatibility bridge during transition
- Location is now part of locality creation through provisional region packets, and those packets now carry descriptor metadata in spatialpayload.scopeddescriptor, but provider-backed normalization, richer external refs, and broader equivalence handling remain later work
- legacy nation | region | city | district locality levels remain current compatibility buckets rather than the full future locality model
- locality ancestry is now planned from ordered broad-to-narrow path entries, and sparse paths are valid current behavior; absolute depth is still projection-only and is not stored on packets

- locality creation now has a real non-mutating review seam, but the home-branch inclusion checklist shown during review is still preview-only and does not yet alter stored residence relation semantics
- locality confirm now applies home, association, follow, and main visibility choices through one composite runtime write seam; claimed main scope-display state now persists through Preference.element packets while guest/session behavior remains compatibility state
- associated and followed parent-context display now persists through the same claimed-actor signed Preference.element path, with guest compatibility fallback still session/cookie-based
- same-name locality matches in other parent branches no longer need to suppress the create-path handoff, but orphan existing-scope remounting is still intentionally unresolved rather than silently repaired in this phase
- generic abstract assembly creation and richer locality governance are still later work

Explorer caveats

- Packet Explorer is read-only for packet mutation
- Packet Explorer Search, Import, and Export are live
- Packet Explorer verification is now live for local validation, imported report reading, and truthful layered status display
- Packet Explorer verification is now revision-aware and stale-aware for the currently preferred revision, but richer revalidation workflows remain unsupported
- Packet Explorer Import now keeps recent import-report history visible inside the Import workspace, but that history currently reflects the latest local report per imported artifact identity rather than a full event-by-event ledger; richer filtering, artifact lineage browsing, and bundle-native history are still later work
- fork or adapt execution, diff, and compare are not live yet
- validation now distinguishes validate first, validate after, and don't validate, but local trust still remains node-local rather than peer-weighted
- packet verification reports and import reports now exist, but richer provenance surfacing, peer-trust weighting, and broader report semantics remain next seams
- a dedicated validation workflow screen is intentionally unsupported
- official versus unofficial initiative filtering is not active product behavior
- initiative-version subscription behavior is not active product behavior
- the generic scope graph is now decoupled from OWA-specific initiative-anchor policy lookup, but broader initiative and locality projection cleanup is still incomplete

Moderation and governance caveats

- moderation workflows are not yet implemented as live user-facing policy systems
- current disabled controls should be read as placeholders, not as hidden active features
- the first high-friction Nexus surfaces now attach compact inline explainers to disabled or partial controls so users can see whether a seam is coming soon, read-only, or partial
- proposals, votes, and decisions are present in packets and read surfaces, but governance is not yet a fully interactive trusted workflow

Provisional guidance

The following areas should still be treated as provisional:

- the shell-level early-access gate is now an honest entry warning, but it is not yet a tutorial, release-note system, or scope-aware onboarding flow
- the new feature-status registry and popover system currently covers Explorer, Votes, and Library first; broader surface rollout is still pending
- deeper trust weighting and moderation consequences
- vote execution, delegation, and propagation semantics
- long-term initiative filtering and visibility modes
- packet actions that currently appear as disabled placeholders
- hardening the remaining Preference trusted runtime connector path before applying the same procedure to other packet types
- any architecture or theory in the implementation guide that is not already represented in executable code